INTRODUCTION

The problem I have implemented is an automatic event planner system to help a university society choose a set of activities that has the highest enjoyment value while complying with time constraints.
The event planner system is able to take an input of a list of activities, each activity having: a name, time required for the activity, cost of the activity and enjoyment value.
The objective of the coursework is to implement brute force and dynamic programming algorithms to find the best set of activities. The chosen constraint for both algorithms is time, ignoring budget but still calculating and printing the total budget and cost.

DESIGN AND APPROACH

Program Structure

My program structure is split into 5 modules, 3 of which are functions and 2 procedures.
read_input(filename) is a function which takes the filename as a parameter. It then opens this file in read mode, loads the activities in this file into a dictionary and returns this dictionary along with the number of activities in the file (n), the time constraint (t) and the budget constraint (b) which were also in the file.
brute_force_solver(acts, t) is a function which takes the activities dictionary (acts) and time (t) as parameters. It then finds the list of activities with the highest enjoyment value that meets the constraint of time (t) and returns from that list of activities, the activities, enjoyment, time, cost and execution time.
Dp_solver(activities, t) is a function which takes the activities dictionary (activities) and time (t) as parameters. It then finds the list of activities with the highest enjoyment value that meets the constraint of time and returns from the list, the activities, enjoyment, time, cost and execution time. It does this in a more efficient manner than brute_force_solver() due to the use of the nested modules total_enjoyment(t, weights, enjoyment), selected_activities(t, weights, dp) and run_dp(activities, t).
Print_results(input_file, selected_activities_bf, total_enjoyment_bf, total_time_bf, total_cost_bf, max_time, max_budget, exec_time_bf, selected_activities_dp, total_enjoyment_dp, total_time_dp, total_cost_dp, exec_time_dp) is a procedure which takes the name of the input file and all the information about the brute force and dynamic programming solvers as parameters. It then prints the information in the correct and clear format by accessing elements in the dictionaries selected_activities_bf and selected_activities_dp.
main() is a procedure which calls the modules read_input(), brute_force_solver(), dp_solver() and print_results.

Data Structures Used

The main data structure used in order to store the activities list was a python list of dictionaries. The dictionary was formatted as: {name, time, cost, enjoyment}. This was chosen so that the attributes of each activity could be easily accessible through methods such as .get(). For example: name = act.get("name"). The list could also be iterated through easily which is especially useful when outputting the activities in print_results().
When itertools.combinations() is used in the brute force solver, the tuple data type is also used as every combination of activities is created in tuples. This is useful to find the activities with the highest enjoyment values as dictionaries can be stored in tuples. Tuples can be used as the combinations of activities can be immutable.

Assumptions/Simplifications Made

Time is assumed to be the main constraint as opposed to budget although budget is tracked and output.
If multiple groups of activities that meet the time constraint have the same number of enjoyments, the activity group with the lowest amount of time is chosen

ALGORITHMS

Brute Force Pseudocode

```
FUNCTION BRUTE_FORCE_SOLVER(activities, T)
        Best_enjoyment = -1
        Best_activities = []
        Best_time = 999
        Best_cost = 0
        FOR subset_size FROM 0 to n:
                FOR subset in all combinations of activities of size subset_size:
                        total_time = SUM of "time"s in subset
                        IF total_time > T:
                                continue
                        ENDIF
                        total_enjoyment = SUM of "enjoyment"s in subset
                        IF total_enjoyment > best_enjoyment:
                                best_enjoyment = total_enjoyment
                                best_time = total_time
                                best_activities = subset
                                best_cost = SUM of "cost"s in subset
                        ENDIF
        RETURN best_activities, best_enjoyment, best_time, best_cost
ENDFUNCTION
```

Dynamic Programming Pseudocode

```
FUNCTION dp_solver(activities, t)
    FUNCTION total_enjoyment(t, weights, enjoyment)
        n = length(weights)
        dp = 2D array of 0's
        FOR i FROM 1 TO n:
            FOR c FROM 0 TO t:
                w = weights[i-1]
                val = enjoyment[i-1]
                IF b>c:
                    dp[i][c] = dp[i-1][c]
                ELSE:
                    dp[i][c] = max(dp[i-1][c], val + dp[i-1][c-w])
                ENDIF
        RETURN dp[n][t], dp
    ENDFUNCTION

    FUNCTION selected_activities(t, weights, dp)
        n = length(weights)
        chosen = []
        i = n
        c = t
        WHILE i>0 AND c>=0 :
            IF dp[i][c] = dp[i-1][c]:
                i = i - 1
            ELSE:
                APPEND (i-1) TO chosen
                c = c – weights[i-1]
                i = i-1
            ENDIF
        ENDWHILE
        REVERSE chosen
        RETURN chosen
    ENDFUNCTION

    FUNCTION run_dp(activities, t)
        weights = LIST of times
        enjoyment = LIST of enjoyment values
        start = START timer
        best, dp = CALL total_enjoyment(t, weights, enjoyment)
```

```
            chosen_indices = CALL selected_activities(t, weights, dp)
            chosen_acts = LIST of activities in chosen_indices
            total_time = SUM of "time"s in chosen_acts
            total_cost = SUM of "cost"s in chosen_acts
            execution_time = STOP timer - start
            RETURN chosen_acts, best, total_time, total_cost, execution_time
        END FUNCITON
        RETURN run_dp(activities, t)
ENDFUNCTION
```

Brute Force Logic and Approach

For the brute force algorithm, initially the variables are all set to certain values. Best_enjoyment is set to -1 so that any subset of activities that satisfy the time constraint will cause the variables to change. Best_enjoyment will then also change whenever a better subset is found.

Best_activities is set to an empty list as lists are mutable and can therefore be changed which is good because the list of the best activities subset will need to be updated as the combinations are iterated through. Best_time is set to 999 as any comparison of time should have it set as high as possible as a lower time is better. Best_cost is set to 0 as it is an integer and will be updated later.

Each subset size is iterated through as every amount of activities should be tested. Each subset is iterated through as every subset for each amount of activities should be tested.

Total_time is temporarily set to the sum of time values for all activities in the tested subset and then this is compared with the T value as it is the constraint for time which the subset must satisfy.

If the time value is too big the iteration of the loop is ended and the next subset is tested. If not, then the total enjoyment of the subsets activities is compared to the total enjoyment of the best subset so far. If this new subset has a better enjoyment value then the values for best enjoyment, time, activities and cost are updated to those from the current subset. Once all iterations have been tested, the values for that subset are returned.

Dynamic Programming Logic and Approach

For the dynamic programming algorithm, first the run_dp() function is called in the return line.

In run_dp() a list called weights is made for the time values aswell as a list called enjoyment for the enjoyment values. This is done as the other functions only require those values from the activities dictionaries.

total_enjoyment() is then called. In this the dynamic programming table is created called dp with n+1 rows and t+1 columns. The table is then filled row by row with the time/weight values as w and also with the enjoyment values as val. The time (w) is then compared to the limit(c) to see if the activity satisfies time. if bigger then the cell above is copied to that cell. If smaller then the cell with the higher enjoyment is copied to both. When the table is complete, the bottom right value (the highest enjoyment) is returned.
Then selected_activities() is called. This goes through the dp table starting in the bottom right and checks if the cell value is the same as the one above. If it different then that activity was chosen and is appended to the chosen list. Once all rows are checked then the chosen list is reversed and returned.
Then the chosen list is used to find the activities dictionaries which is then used to find the total_time and total_cost.
These values are then all returned at the end of run_dp() meaning they are returned for dp_solver() also.

Brute Force Time/Space Complexity

The time complexity for the brute force algorithm is exponential as it iterates through 2^n subsets. For each subset, total values for time are calculated.
The space complexity for the brute force algorithm is fairly low as the most that is stored is the current and best subsets as dictionaries.

Dynamic Programming Time and Space Complexity

The time complexity for the dynamic programming algorithm is polynomial as its time complexity is n x t as the size of the dp table is n x t and it is iterated through once.
The space complexity for the dynamic programming algorithm is n x t as the dp table is stored.

TESTING AND RESULTS.

How Brute Force Algorithm was Tested

Brute force algorithm was tested by running a reduced version of the program where the dynamic programming algorithm had not yet been implemented. This allowed focus on the output of the brute force algorithm. The test cases that were used for the brute force algorithm were input_small, input_medium and input_large only. This is because the testing of other files was not possible due to the exponential complexity of the algorithm. If input_100 or more was to be tested the test would take too long. The program was tested with the following input to the terminal: python Code/event_planner.py "Input_Files/Sample Input Files-20260121/input_small.txt". This syntax was continued for input_medium.txt and input_large.txt.

Results of Brute Force Algorithm

When:
python Code/event_planner.py "Input_Files/Sample Input Files-20260121/input_small.txt"
is input into the terminal, the result is as follows:

```
========================================
EVENT PLANNER - RESULTS
========================================

Input File:  Input_Files/Sample Input Files-20260121/input_small.txt
Available Time: 10
Available Budget: 200

--- BRUTE FORCE ALGORITHM ---
Selected Activities:
  - Game-Night (3 hours, £80, enjoyment 120)
  - Museum-Trip (4 hours, £100, enjoyment 150)
  - Pizza-Workshop (2 hours, £60, enjoyment 100)

Total Enjoyment: 370
Total Time Used: 9 hours
Total cost: £240

Execution Time: 3.91e-05 seconds
```

When:
python Code/event_planner.py "Input_Files/Sample Input Files-20260121/input_medium.txt"
is input into the terminal, the result is as follows:

```
========================================
EVENT PLANNER - RESULTS
========================================

Input File:  Input_Files/Sample Input Files-20260121/input_medium.txt
Available Time: 15
Available Budget: 300

--- BRUTE FORCE ALGORITHM ---
```

Selected Activities:
  - Film-Screening (3 hours, £30, enjoyment 90)
  - Art-Workshop (2 hours, £70, enjoyment 95)
  - Bowling (3 hours, £80, enjoyment 100)
  - Laser-Tag (2 hours, £90, enjoyment 130)
  - Cooking-Class (3 hours, £75, enjoyment 105)
  - Escape-Room (2 hours, £85, enjoyment 115)

Total Enjoyment: 635
Total Time Used: 15 hours
Total cost: £430

Execution Time: 0.0032151 seconds

When:
python Code/event_planner.py "Input_Files/Sample Input Files-20260121/input_large.txt"
is input into the terminal, the result is as follows:

==========
EVENT PLANNER - RESULTS
==========

Input File: Input_Files/Sample Input Files-20260121/input_large.txt
Available Time: 20
Available Budget: 500

--- BRUTE FORCE ALGORITHM ---
Selected Activities:
  - Orientation-Walk (1 hours, £10, enjoyment 30)
  - Wine-Tasting (2 hours, £100, enjoyment 150)
  - Rock-Climbing (4 hours, £110, enjoyment 160)
  - Pottery-Class (3 hours, £85, enjoyment 125)
  - Dance-Workshop (2 hours, £50, enjoyment 90)
  - Kayaking (5 hours, £130, enjoyment 190)
  - Comedy-Show (3 hours, £80, enjoyment 135)

Total Enjoyment: 880
Total Time Used: 20 hours
Total cost: £565

Execution Time: 30.8476845 seconds

How Dynamic Programming Algorithm was Tested

The dynamic programming algorithm was tested on the final version of the code for event_planner.py. This meant that the results of the brute_force algorithm were also included but they have been removed in the results for clarity and focus on the dynamic programming algorithms results. The test cases that were used for the dynamic programming algorithm were input_small, input_medium and input_large. This is because the code is designed to run alongside the brute force algorithm and the usage of inputs with more activities would cause the code to never finish running due to the exponential growth of the brute force algorithm.

Results of Dynamic Programming Algorithm

When:
python Code/event_planner.py "Input_Files/Sample Input Files-20260121/input_small.txt"
is input into the terminal, the result is as follows:

```
========================================
EVENT PLANNER - RESULTS
========================================

Input File:  Input_Files/Sample Input Files-20260121/input_small.txt
Available Time: 10
Available Budget: 200

--- DYNAMIC PROGRAMMING ALGORITHM ---
Selected Activities:
  - Game-Night (3 hours, £80, enjoyment 120)
  - Museum-Trip (4 hours, £100, enjoyment 150)
  - Pizza-Workshop (2 hours, £60, enjoyment 100)

Total Enjoyment: 370
Total Time Used: 9 hours
Total cost: £240

Execution Time: 1.35e-05 seconds

========================================
```

When:
python Code/event_planner.py "Input_Files/Sample Input Files-20260121/input_medium.txt"
is input into the terminal, the result is as follows:

```
========================================
EVENT PLANNER - RESULTS
========================================

Input File:  Input_Files/Sample Input Files-20260121/input_medium.txt
Available Time: 15
Available Budget: 300

--- DYNAMIC PROGRAMMING ALGORITHM ---
Selected Activities:
  - Film-Screening (3 hours, £30, enjoyment 90)
  - Art-Workshop (2 hours, £70, enjoyment 95)
  - Bowling (3 hours, £80, enjoyment 100)
  - Laser-Tag (2 hours, £90, enjoyment 130)
  - Cooking-Class (3 hours, £75, enjoyment 105)
  - Escape-Room (2 hours, £85, enjoyment 115)

Total Enjoyment: 635
Total Time Used: 15 hours
Total cost: £430

Execution Time: 3.14e-05 seconds

========================================
```

When:
python Code/event_planner.py "Input_Files/Sample Input Files-20260121/input_large.txt"
is input into the terminal, the result is as follows:

```
========================================
EVENT PLANNER - RESULTS
========================================

Input File:  Input_Files/Sample Input Files-20260121/input_large.txt
Available Time: 20
```

Available Budget: 500

--- DYNAMIC PROGRAMMING ALGORITHM ---
Selected Activities:
  - Orientation-Walk (1 hours, £10, enjoyment 30)
  - Wine-Tasting (2 hours, £100, enjoyment 150)
  - Rock-Climbing (4 hours, £110, enjoyment 160)
  - Pottery-Class (3 hours, £85, enjoyment 125)
  - Dance-Workshop (2 hours, £50, enjoyment 90)
  - Kayaking (5 hours, £130, enjoyment 190)
  - Comedy-Show (3 hours, £80, enjoyment 135)

Total Enjoyment: 880
Total Time Used: 20 hours
Total cost: £565

Execution Time: 5e-05 seconds

==========

Compare Results and Performance

For each of the 3 test cases, both the brute force algorithm and dynamic programming algorithm returned the same selection of activities, enjoyment, time used and cost. This shows that both algorithms performed their function correctly.
Time taken for each test case:
Small
- Brute Force: 0.0000391 seconds
- Dynamic programming: 0.0000135 seconds

Medium
- Brute Force: 0.0032151 seconds
- Dynamic programming: 0.0000314 seconds

Large
- Brute Force: 30.8476845 seconds
- Dynamic programming: 0.00005 seconds

These results show that dynamic programming was faster for all test cases and was significantly faster than brute force in the large test case, being over 600,000 times quicker. This is due to the exponential growth of the brute force algorithm compared to the polynomial growth of the dynamic programming algorithm. The performance of the dynamic programming algorithm is clearly better.

The limitations encountered were that I could not test the brute force or dynamic programming algorithm on larger test cases such as input_1000 due to the exponential growth of the brute force algorithm making it so the running of the program would take a very long time. Because the dynamic programming algorithm could not be run independently easily, I could not test this on input_1000 either. This is due to the algorithms being poorly designed as they are run one by one in main() and so both must be run at the same time.
Another limitation of the algorithm is that it only accounts for time as a constraint and does not consider budget.

References

No references used.